

Project Report

Career Interest Recommender System

Course/Subject: Python essentials

Language: Python

Project Focus : Interactive Decision Support Systems & Rule-Based Analytics

Prepared By: Akshat Chhitarka

Registration Number: 26BCE11441

Date of Submission: 30 September 2026

• Introduction:

Choosing a specialization within computer science and information technology can be overwhelming for early-stage students and self-learners. With multiple overlapping domains—such as Artificial Intelligence, Web Development, Cybersecurity, Data Science, UI/UX Design, and Cloud/DevOps—learners frequently struggle to align their personal interests with potential career paths.

The **Career Interest Recommender System** is an interactive, light-weight diagnostic tool designed to address this challenge. Built using standard Python, the system guides users through a targeted questionnaire to evaluate their natural preferences, calculates match scores across six primary tech domains, identifies top career recommendations, and presents tailored learning paths alongside future job outcomes.

• Problem Statement

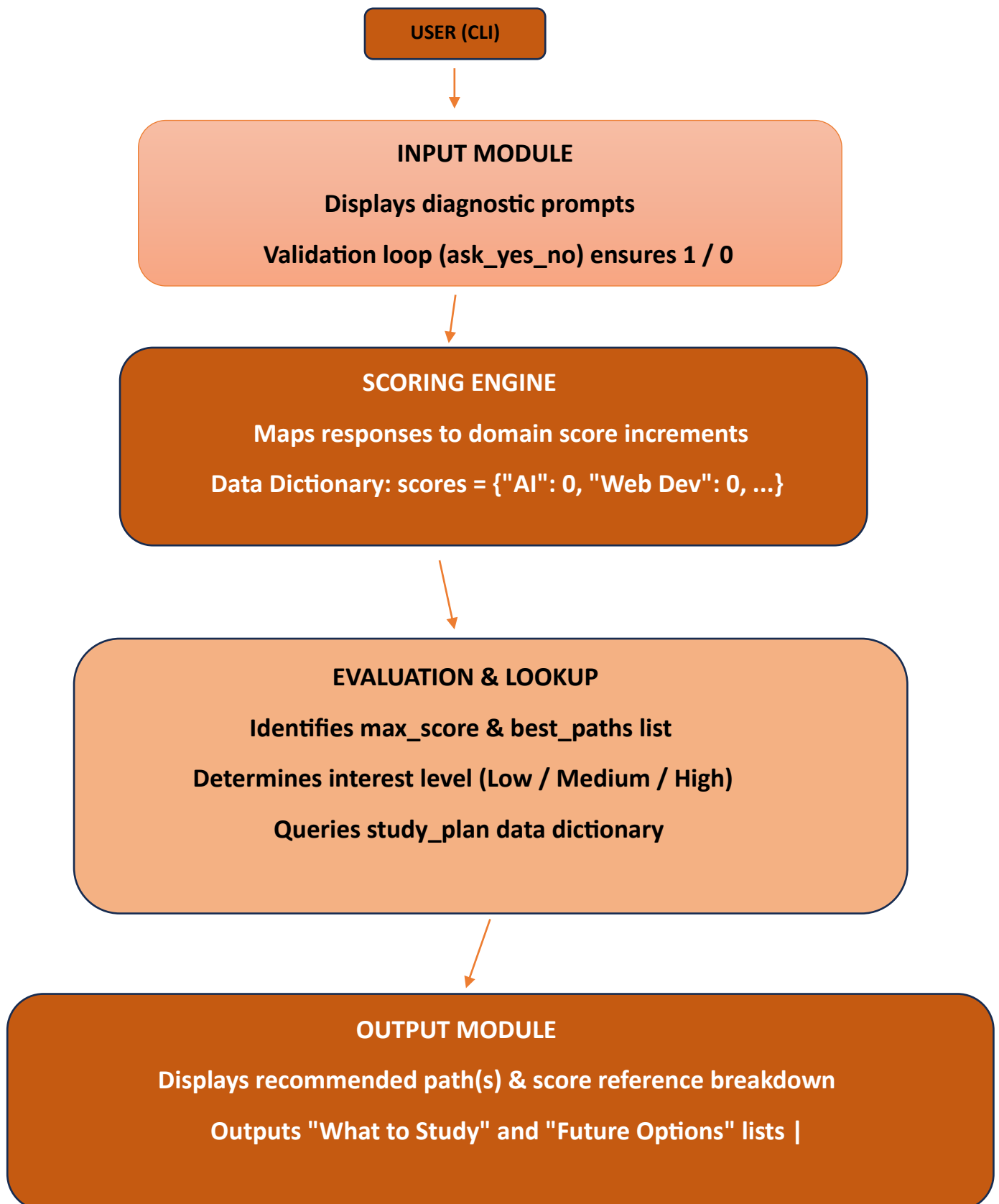
Tech-aspiring students often face two primary hurdles:

1. **Decision Paralysis:** The tech industry features numerous distinct specializations, making it difficult to decide where to focus initial effort.
2. **Lack of Direction:** Even when students discover an area of interest, they often lack immediate clarity on core fundamental concepts to study first and what job titles align with that field.

Existing career guidance platforms can be overly complex, requiring lengthy assessments or paid subscriptions. There is a need for a straightforward, rule-based interactive program that provides immediate, targeted clarity regarding tech specializations and foundation learning roadmaps.

System Architecture:

- The project follows a centralized procedural design built around structured memory components and an interactive CLI interface loop.



Workflow Steps

Start Program

Initialize Scores & Study Plan

Prompt Question i

Input Validation: Is input '0' or '1'?

If NO :- Print Warning Message, then end

If YES :- Process Score Weights

[NOTE:- (Loop for all 6 questions)]

Evaluate Max Score

Map Qualitative Interest

Print Recommendations & Study Roadmap Details

END

Design Decisions & Rationale

1. In-Memory Data Structures over File/DB Storage:

- *Rationale:* The data domain (6 tech fields, short guidance lists) is static and compact. Storing rules directly in dictionary constructs keeps the application self-contained, portable, zero-dependency, and lightweight.

2. Binary Choice Strategy (1/0 Inputs):

- *Rationale:* Binary questions eliminate ambiguous fuzzy states, simplifying score processing while keeping user interaction fast and intuitive.

3. Multi-Category Weighting Strategy:

- *Rationale:* Single responses increment multiple complementary categories (e.g., Q1 adds +2 to AI/ML and Data Science, but +1 to Cybersecurity). This reflects real-world overlap where core competencies (like math and analytical logic) benefit multiple fields.

4. Tie-Handling via List Comprehension:

- *Rationale:* Using `best_paths = [name for name, val in scores.items() if val == max_score]` gracefully handles ties when a user has multi-disciplinary interests, recommending all top-scoring paths instead of arbitrarily choosing one.

Implementation Details

The implementation is written in standard **Python**.

Core Architecture Components:

- **scores Dictionary:** Initializes base scores to zero for six domains:
 1. AI & Machine Learning
 2. Web Development
 3. Cybersecurity
 4. Data Science
 5. UI/UX Design
 6. Cloud & DevOps
- **study_plan Dictionary:** Serves as a internal knowledge repository mapping each domain key to a nested dictionary containing "study" (list of foundational concepts) and "Future Occupation" (list of targeted job roles).
- **Input Helper (ask_yes_no):** Implements a continuous while True evaluation loop that enforces strict input constraints ("0" or "1"), ensuring robust user error recovery.
- **Rule Engine Execution:** Sequential execution evaluates responses Q_1 through Q_6 , adding weighted values directly to domain scores.

Screenshots / Results:-

```
career_predictor.py > ...
5      },
6      "Cybersecurity": {
7          "study": [
8              "computer networks and OS basics",
9              "Linux commands and system security basics",
10             "Common attacks (phishing, malware, SQL injection)",
11             "Future Occupation": [
12                 "Cybersecurity Analyst",
13                 "Ethical Hacker",
14                 "Security Engineer"
15             ]
16         },
17         "Data Science": {
18             "study": [
19                 "Python, statistics, and data analysis",
20                 "Pandas, NumPy, and data visualization",
21                 "Basic machine learning algorithms",
22                 "Future Occupation": [
23                     "Data Analyst",
24                     "Data Scientist",
25                     "Business Intelligence Engineer"
26                 ]
27             },
28             "UI/UX Design": {
29                 "study": [
30                     "Design principles and color theory",
31                     "Wire framing tools (Figma, Adobe XD basics)",
32                     "User research and usability testing",
33                     "Future Occupation": [
34                         "UI Designer",
35                         "UX Designer",
36                         "Product Designer"
37                     ]
38                 }
39             }
40         }
41     },
42     "Cloud & Devops": {
43         "study": [
44             "Basics of Linux and command line",
45             "Intro to cloud platforms (AWS, Azure, GCP)",
46             "Version control (Git) and CI/CD basics",
47             "Future Occupation": [
48                 "Cloud Engineer",
49                 "Devops Engineer",
50                 "Site Reliability Engineer"
51             ]
52         }
53     }
54 }
```

```

54     },
55     "Cloud & Devops": {
56         "study": [
57             "Basics of Linux and command line",
58             "Intro to cloud platforms (AWS, Azure, GCP)",
59             "Version control (Git) and CI/CD basics",
60             "Future Occupation": [
61                 "Cloud Engineer",
62                 "Devops Engineer",
63                 "Site Reliability Engineer"
64             ]
65         }
66     }
67 }

help: get 0/1 safely
ask_yes_no(question):
while True:
    ans = input(question + " ")
    if ans in ["0", "1"]:
        return int(ans)
    print("Please enter only 1 for Yes or 0 for No.")

# short questions
= ask_yes_no("Do you enjoy solving logical or math problems? (1/0)")
q1:=1:
scores["AI & Machine Learning"] += 2
scores["Data Science"] += 2
scores["Cybersecurity"] += 1

= ask_yes_no("Do you like creating or designing visually appealing things? (1/0)")
q2:=1:
scores["UI/UX Design"] += 2
scores["Web Development"] += 1
```

```
career_predictor.py > ...
5      },
6      "Cybersecurity": {
7          "study": [
8              "computer networks and OS basics",
9              "Linux commands and system security basics",
10             "Common attacks (phishing, malware, SQL injection)",
11             "Future Occupation": [
12                 "Cybersecurity Analyst",
13                 "Ethical Hacker",
14                 "Security Engineer"
15             ]
16         },
17         "Data Science": {
18             "study": [
19                 "Python, statistics, and data analysis",
20                 "Pandas, NumPy, and data visualization",
21                 "Basic machine learning algorithms",
22                 "Future Occupation": [
23                     "Data Analyst",
24                     "Data Scientist",
25                     "Business Intelligence Engineer"
26                 ]
27             },
28             "UI/UX Design": {
29                 "study": [
30                     "Design principles and color theory",
31                     "Wire framing tools (Figma, Adobe XD basics)",
32                     "User research and usability testing",
33                     "Future Occupation": [
34                         "UI Designer",
35                         "UX Designer",
36                         "Product Designer"
37                     ]
38                 }
39             }
40         }
41     },
42     "Cloud & Devops": {
43         "study": [
44             "Basics of Linux and command line",
45             "Intro to cloud platforms (AWS, Azure, GCP)",
46             "Version control (Git) and CI/CD basics",
47             "Future Occupation": [
48                 "Cloud Engineer",
49                 "Devops Engineer",
50                 "Site Reliability Engineer"
51             ]
52         }
53     }
54 }
```

```

q3 = ask_yes_no("Are you curious about how websites and apps are built? (1/0)")
if q3==1:
    scores["Web Development"] += 2
    scores["Cloud & Devops"] += 1

q4 = ask_yes_no("Do you like finding patterns in data or charts? (1/0)")
if q4==1:
    scores["Data Science"] += 2
    scores["AI & Machine Learning"] += 1

q5 = ask_yes_no("Are you interested in protecting systems from hackers? (1/0)")
if q5==1:
    scores["Cybersecurity"] += 2
    scores["Cloud & Devops"] += 1

q6 = ask_yes_no("Do you enjoy working with tools, automation, or cloud platforms? (1/0)")
if q6==1:
    scores["Cloud & Devops"] += 2
    scores["AI & Machine Learning"] += 1

# Find best paths
max_score = max(scores.values())
best_paths = [name for name, val in scores.items() if val==max_score]

# Interest level
if max_score <= 2:
    interest_level = "Low ☐ You are still exploring different areas."
elif max_score <= 4:
    interest_level = "Medium ☐ This area matches some of your interests."
else:
    interest_level = "High ☐ This area strongly matches your interests!"

print("\n==== Result =====\n")

```

```

✓ if len(best_paths)==1:
    print("Your Recommended Career Path is:")
✓ else:
    print("You match well with multiple career paths:")

✓ for path in best_paths:
    print("->", path)

    print("\nInterest level:", interest_level)

    print("\nScores for reference:")
    ✓ for name, val in scores.items():
        print(f"{name}: {val}")

# Show what to study and future options for each best path
print("\n==== What You Should Study & Future Options =====\n")

✓ for path in best_paths:
    info = study_plan[path]
    print("Career Path:", path)
    print("What you should study first:")
    ✓ for item in info["study"]:
        print("  -", item)
    print("Future career options:")
    ✓ for job in info["Future occupation"]:
        print("  -", job)
    print() # blank line between paths

print("Note: This is a simple guide. Explore online courses and resources for deeper learning")

```

Testing Approach:-

- Testing was conducted manually across various response patterns to ensure scoring accuracy and input stability.

TEST CASE	Input Sequence ($Q_1 \dots Q_6$)	Expected Top Result	Output Validation Status
1: Pure AI Focus	1, 0, 0, 1, 0, 1	Tie: AI & ML (4), Data Science (4)	PASS
2: Pure Design Focus	0, 1, 1, 0, 0, 0	UI/UX Design (2), Web Dev (3)	PASS
3: Security & Cloud	0, 0, 0, 0, 1, 1	Cloud & DevOps (3), Cybersecurity (2)	PASS
4: Low Interest	0, 0, 0, 0, 0, 0	All 0 (Tie all), Level: "Low"	PASS
5: Invalid Input	abc $\rightarrow$ 5 $\rightarrow$ 1	Trapped error message, accepts 1	PASS

Challenges Faced:

1. Handling Dual Specializations (Ties):

- *Challenge:* Early design drafts assumed a single maximum score. Users with broad interests resulted in arbitrarily selected recommendations.

- *Resolution:* Implemented dynamic list filtering ([name for name, val in scores.items() if val == max_score]) to present all tied recommendations alongside customized guidance for each.

2. Preventing CLI Crashes on Invalid Input:

- *Challenge:* Direct integer conversion (int(input())) crashed when users typed string letters or empty inputs.
- *Resolution:* Built ask_yes_no() to validate raw string inputs before casting to integers.

Learnings & Key Takeaways:

- **Data Structure Selection Matters:** Using dictionary lookups for domain scores and study tracks produced far cleaner code than nested if-else blocks.
- **User Input Sanitization:** Defensive validation loops are critical for CLI tools to prevent unexpected runtime terminations.
- **Rule-Based Decision Systems:** Learned how weighted scoring models can translate qualitative answers into actionable recommendations.

Future Enhancements:

- **Graphical User Interface (GUI):** Transition from a command-line utility to a web app using Streamlit, Flask, or Tkinter.
- **Expanded Question Bank & Weighted Scoring:** Add Likert-scale responses (1 to 5) instead of binary (0/1) choices for finer nuance.
- **External Resource Links:** Integrate direct links to free learning platforms (e.g., Coursera, edX, freeCodeCamp) matching recommended topics.

- **User Progress Persistence:** Add local database support (SQLite) to store past assessment runs and track changing user interests over time.

References:

1. Python Software Foundation. *Python 3 Documentation - Dictionaries and Data Structures*.
<https://docs.python.org/3/tutorial/datastructures.html>
2. Pressman, R. S. *Software Engineering: A Practitioner's Approach*. McGraw-Hill Education.
3. ACM Computing Curricula Guidelines. *Curriculum Guidelines for Undergraduate Degree Programs in Computer Science*.

THANKYOU!!!